

BỘ THÔNG TIN VÀ TRUYỀN THÔNG
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

-----o0o-----

INFORMATION SYSTEM ANALYSIS AND DESIGN



ASSIGNMENT 06

Giảng viên hướng dẫn	: Trần Đình Quế
Họ và tên sinh viên	: Nguyễn Tiến Đạt
Mã sinh viên	: B23DCCE017
Lớp	: E23TTNT02

Hà Nội – 2026

Contents

1. Introduction;	3
2. Problem Description;	3
3. Requirements Analysis;	4
3.1. Actor	4
3.2. Functional Requirements	4
3.3. Non-Functional Requirements	5
3.4. Inputs and Outputs	6
4. Use Case Model;	6
4.1. Use Case Descriptions	6
4.2. Use Case Relationships	7
4.3. Main Scenario: Search by Voice	8
5. Three-Layer Architecture;	9
5.1. Presentation Layer	9
5.2. Application / Intelligence Layer	9
5.3. Data Layer	10
5.4. Dependencies and Processing	10
6. UML Sequence Diagram;	13
6.1. Participants	13
6.2. Interaction Sequence	13
6.3. Alternative Flows	15

1. Introduction;

This report presents the analysis, design, and implementation of a prototype multimodal search system for e-commerce. The system supports product searches through text, simulated voice input, and image representations. These inputs follow a unified process: query representation, candidate retrieval, ranking, and result presentation.

The system adopts a three-layer architecture: the Presentation Layer handles user interaction, the Application / Intelligence Layer processes queries and ranks results, and the Data Layer manages product information, images, and feature vectors. UML diagrams created in Visual Paradigm guide the Python implementation.

The prototype uses keyword matching for text search, predefined transcripts for voice search, and artificial feature vectors with cosine similarity for image search. Its purpose is to demonstrate the relationship between software requirements, architectural design, and executable code. Evaluation focuses on search results and prototype limitations rather than production-level AI performance

2. Problem Description;

Customers may search for products using keywords, spoken requests, or reference images. These inputs require different processing methods but should produce a consistent output: a ranked list of relevant products.

The system must support three search scenarios:

- **Text search:** A customer enters a query such as “black running shoes.” The system matches query terms against product information.
- **Voice search:** A customer provides a spoken request such as “find running shoes.” In the prototype, a predefined transcript simulates speech recognition and follows the text-search process.
- **Image search:** A customer provides a sample product image. The prototype uses a predefined feature vector to retrieve products through cosine similarity.

The product dataset contains at least ten products, including identifiers, names, categories, colors, prices, stock quantities, and image references. Search results display product information and ranking scores. Empty inputs, unsupported image samples, and queries with no matching products must receive appropriate feedback.

The broader system also includes order lookup and order details, while the core implementation focuses on product search. The main design challenge is to integrate different input methods into a shared search workflow while maintaining clear separation between user interaction, application logic, and data access.

3. Requirements Analysis;

The requirements define the system's users, supported functions, quality constraints, and expected inputs and outputs. The core prototype focuses on product search, while order search is included in the broader system design.

3.1. Actor

Customer is the primary actor. Customers search for products using text, simulated voice input, or sample images, then inspect the returned results. In the extended scope, they can also search for and view their orders.

3.2. Functional Requirements

ID	Requirement	Scope
FR01	The system shall allow customers to search products using text keywords.	Core
FR02	The system shall process a supplied voice transcript through the text-search workflow.	Core
FR03	The system shall support image-similarity search using predefined image feature vectors.	Core
FR04	The system shall convert supported inputs into a common query structure.	Core

FR05	The system shall retrieve candidate products and rank them by relevance score.	Core
FR06	The system shall display product information and ranking scores, limited to a specified number of results.	Core
FR07	The system shall provide appropriate feedback for empty queries, unsupported images, and searches with no results.	Core
FR08	The system shall allow customers to view details of a selected product.	Core
FR09	The system shall allow customers to find their orders by ID and view order details.	Extension

3.3. Non-Functional Requirements

ID	Requirement
NFR01 — Maintainability	The system shall separate presentation, application logic, and data access. The presentation layer shall not access stored data directly.
NFR02 — Extensibility	Simulated speech and image processing components shall be replaceable without changing the common search interface.
NFR03 — Reliability	Invalid inputs shall produce clear feedback without terminating the application unexpectedly.
NFR04 — Reproducibility	Identical queries and data shall produce the same ranking, including a defined rule for tied scores.
NFR05 — Usability	The interface shall clearly identify the search mode, required input, and returned results.
NFR06 — Portability	The project shall include documented dependencies and execution instructions, without machine-specific file paths.

3.4. Inputs and Outputs

Search Mode	Prototype Input	Processing	Output
Text	Keyword string	Normalization and keyword matching	Ranked products with text-matching scores
Voice	Simulated speech transcript	Transcript processing followed by text search	Ranked products with text-matching scores
Image	Supported sample image	Predefined vector lookup and cosine similarity	Ranked products with similarity scores

The dataset shall contain at least ten products with consistent identifiers and relevant attributes. Image-search vectors shall have matching dimensions and reference valid products. The prototype shall clearly disclose that speech recognition and image feature extraction are simulated.

4. Use Case Model;

The use case model describes the system's functions from the customer's perspective. The system boundary is named **Multimodal E-Commerce Search System**, with **Customer** as the primary actor.

4.1. Use Case Descriptions

Use Case	Description
Search Product	Find relevant products using a supported input method.
Search by Keyword	Find products by matching text keywords against product information.
Search by Voice	Find products using a voice transcript, with speech recognition simulated in the prototype.

Search by Image	Find similar products using predefined image feature vectors.
Search Order	Find an order belonging to the customer using its identifier.
View Product	Display details of a selected product.
View Order	Display details of a selected order, including its status and total.

4.2. Use Case Relationships

Search by Keyword, **Search by Voice**, and **Search by Image** specialize **Search Product** through generalization relationships. Each represents a different way to achieve the same product-search goal.

The Customer is associated with **Search Product**, **Search Order**, **View Product**, and **View Order**. Product and order viewing are modeled as separate user goals. Order-related use cases represent the extended system scope.

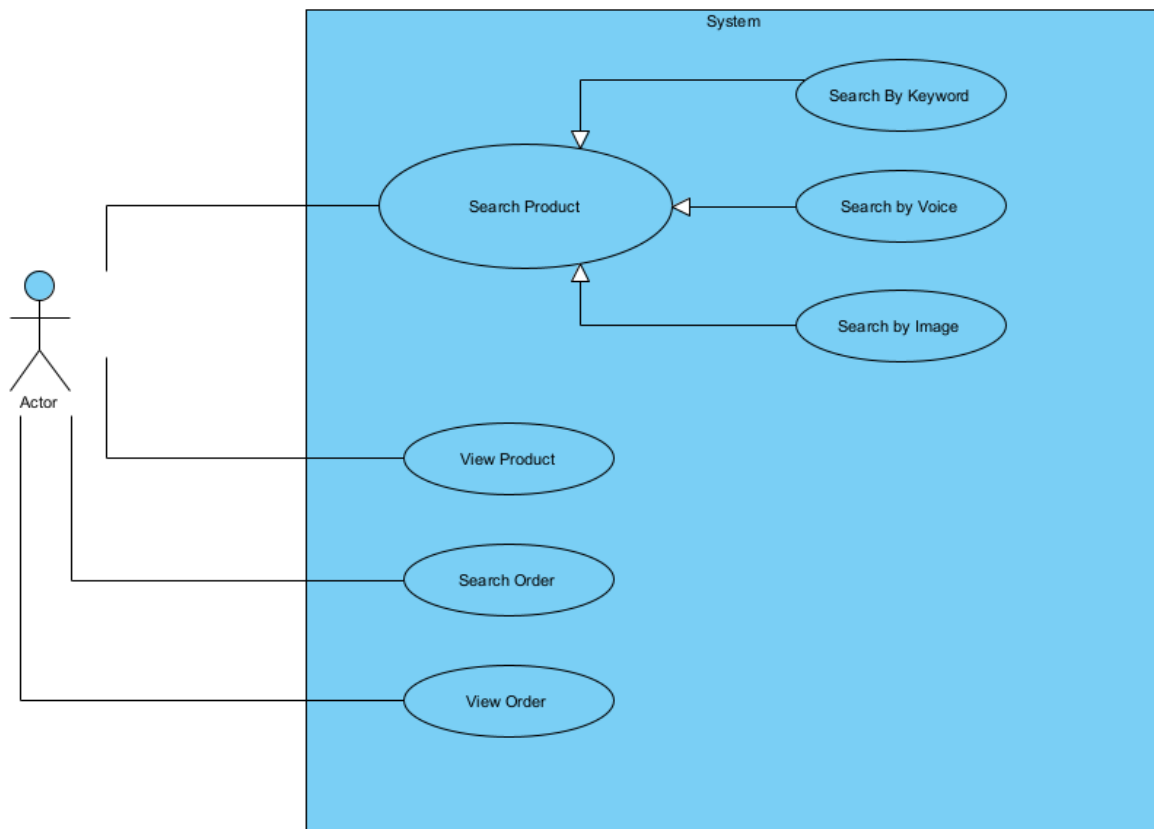


Figure 1. Use case model of the multimodal e-commerce search system.

4.3. Main Scenario: Search by Voice

Item	Description
Primary actor	Customer
Precondition	Product data is available.
Trigger	The customer selects voice search.
Main flow	1. The customer supplies a simulated voice input. 2. The system obtains its transcript. 3. The system constructs a query. 4. Candidate products are retrieved and ranked. 5. Results and scores are displayed.

Alternative flows	An empty transcript produces a validation message. A query with no matches produces a no-results message.
Postcondition	Ranked results or an appropriate message are displayed.
Prototype constraint	Speech recognition is simulated using a supplied transcript.

5. Three-Layer Architecture;

The system follows a three-layer architecture that separates user interaction, application processing, and data management. This separation supports maintainability and allows simulated AI components to be replaced without redesigning the entire system.

5.1. Presentation Layer

The Presentation Layer collects customer input and displays search results. It delegates query processing to the Application Layer and does not directly access stored data.

Component	Responsibility
SearchUI	Coordinate user interaction and submit search requests.
VoiceInput	Collect the transcript used to simulate voice input.
ImageUpload	Accept a supported sample image path.
SearchResultView	Display product information, ranking scores, and feedback messages.

5.2. Application / Intelligence Layer

The Application Layer interprets inputs, retrieves candidate products, and ranks results.

Component	Responsibility
-----------	----------------

QueryService	Normalize input and construct a common query object.
SpeechService	Return a transcript from simulated voice input.
ImageService	Obtain the predefined feature vector associated with a sample image.
SearchService	Coordinate retrieval according to the query type.
RankingService	Sort candidates by score and select the top-k results.

Retrieval and ranking are separate operations: retrieval identifies candidates and computes relevance scores, while ranking determines their final order.

5.3. Data Layer

The Data Layer provides access to product information, order information, images, and feature vectors.

Component	Responsibility
ProductRepository	Provide product retrieval operations.
OrderRepository	Provide order lookup operations in the extended scope.
VectorIndex	Store and provide product feature vectors for similarity calculations.
ProductDatabase	Store product records, implemented using JSON in the prototype.
OrderDatabase	Store order records for the extension.
ImageStorage	Store sample images and their predefined query-vector mappings.

5.4. Dependencies and Processing

Dependencies follow the direction **Presentation** → **Application** → **Data**, with additional dependencies between components within the same layer.

SearchUI uses the input components, invokes application services, and delegates result presentation to SearchResultView. SearchService obtains product data through ProductRepository, accesses product vectors through VectorIndex, and passes scored candidates to RankingService. ImageService accesses ImageStorage to obtain the predefined representation of a supported image.

The architecture represents logical responsibilities. In the Python prototype, multiple presentation components may be implemented within the same module. Order-related components belong to the extended scope.

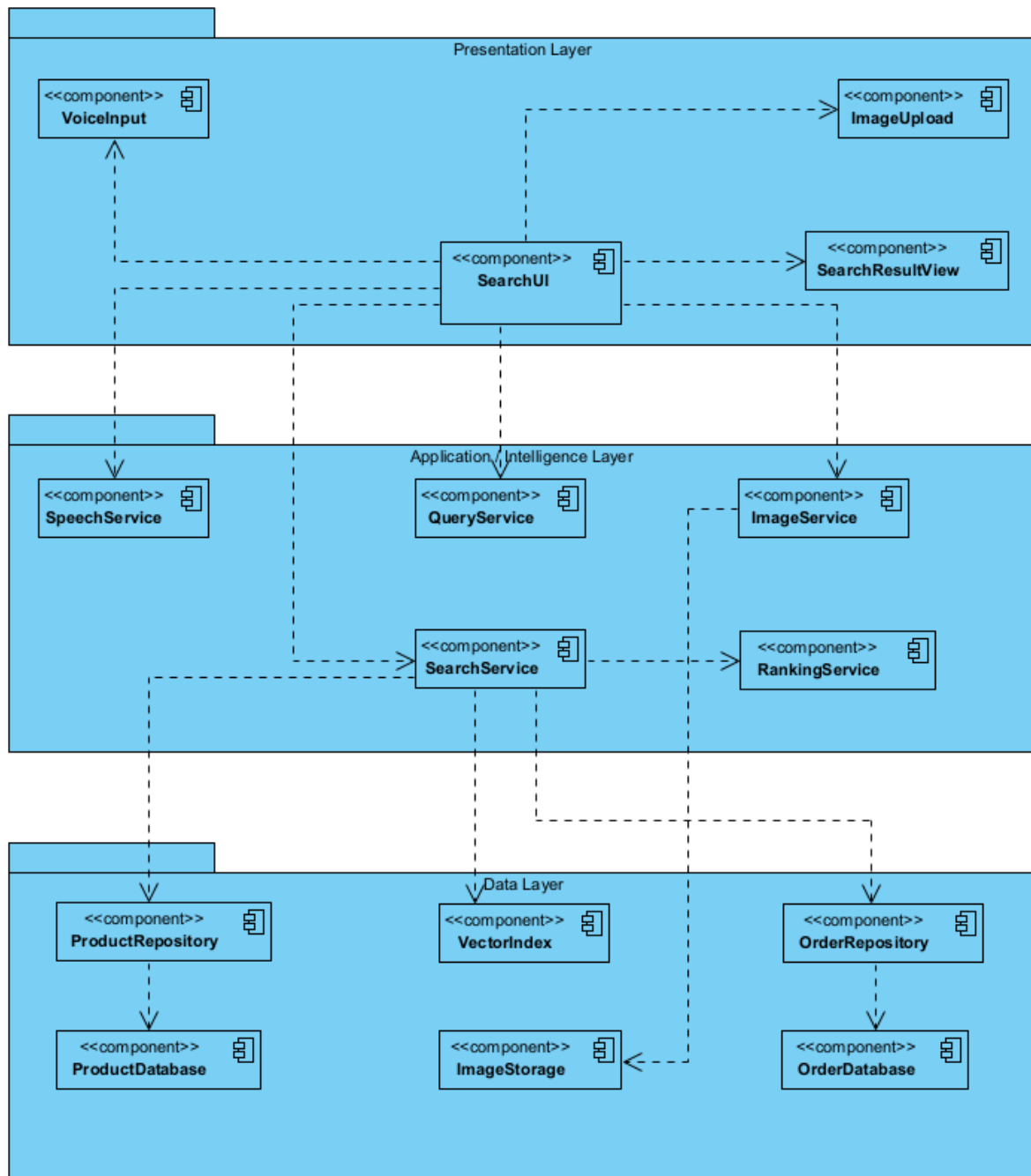


Figure 2. Three-layer architecture of the multimodal e-commerce search system.

6. UML Sequence Diagram;

The sequence diagram describes how system components collaborate when a customer searches for products using voice input. Speech recognition is simulated using a supplied transcript.

6.1. Participants

Participant	Responsibility
Customer	Initiate voice search and provide input.
SearchUI	Coordinate user interaction and service calls.
SpeechService	Return the simulated transcript.
QueryService	Normalize the transcript and construct a query object.
SearchService	Retrieve candidate products and calculate matching scores.
ProductRepository	Provide product records.
RankingService	Sort candidates and select the top-k results.

For readability, input collection and result presentation are represented within SearchUI. Their presentation-layer helper components are omitted from this interaction view.

6.2. Interaction Sequence

Step	Interaction	Description
1	Customer → SearchUI: submitVoice(voiceInput)	The customer submits a simulated voice input.
2	SearchUI → SpeechService: transcribe(voiceInput)	SearchUI requests a transcript.
3	SpeechService → SearchUI: return transcript	SpeechService returns the supplied text as the simulated transcription.

4	SearchUI → QueryService: voice_query(transcript)	SearchUI requests a normalized voice-query object.
5	QueryService → SearchUI: return query	QueryService returns the constructed query.
6	SearchUI → SearchService: search(query, top_k)	SearchUI submits the query and requested result limit.
7	SearchService → ProductRepository: all_products()	SearchService requests the product collection.
8	ProductRepository → SearchService: return products	The repository returns the available product records.
9	SearchService → SearchService: retrieve_text_candidates(query, products)	SearchService identifies matching products and calculates relevance scores.
10	SearchService → RankingService: rank(candidates, top_k)	SearchService requests candidate sorting and top-k selection.
11	RankingService → SearchService: return ranked_results	RankingService returns the highest-ranked candidates.
12	SearchService → SearchUI: return ranked_results	SearchService forwards the final results to the interface.
13	SearchUI → SearchUI: display_results(ranked_results)	SearchUI performs an internal operation to display the ranked products and their scores.

The diagram represents the successful search scenario. The final interaction denotes user-visible output rather than a method executed by the Customer.

Synchronous calls are represented by solid message arrows, while return messages use dashed arrows. Activation bars indicate periods of execution. Candidate retrieval is shown as a self-call within SearchService.

6.3. Alternative Flows

An alt combined fragment distinguishes valid and empty transcripts. If the transcript is empty, SearchUI displays a validation message and does not invoke the search workflow.

After processing a valid query, another alt fragment distinguishes non-empty results from an empty result list. The system either displays ranked products or informs the customer that no matching products were found.

The interaction preserves the three-layer architecture: SearchUI invokes application services, and SearchService accesses product data through ProductRepository. Retrieval and ranking remain separate responsibilities.

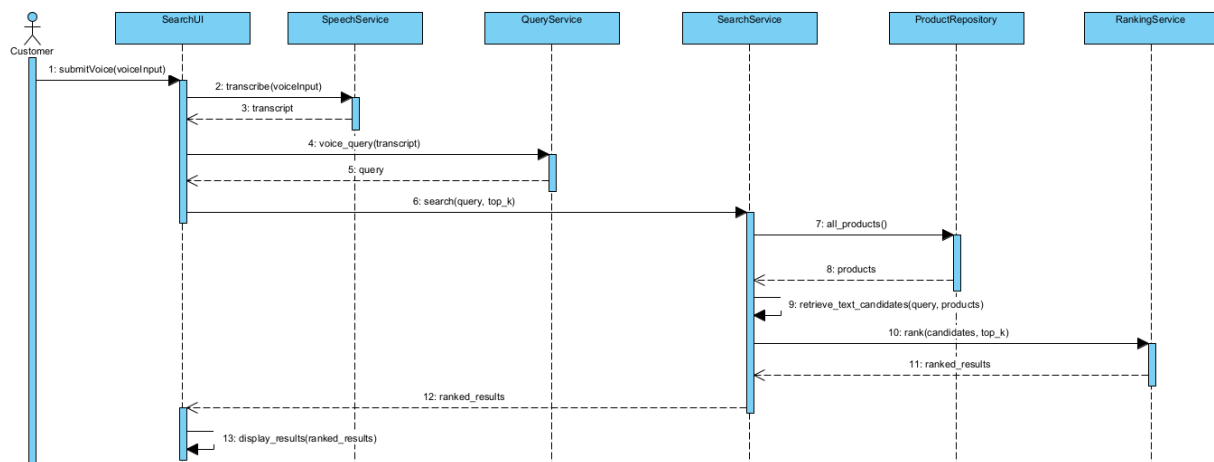


Figure 3. Sequence diagram for product search using simulated voice input.